Re-InnovateX — Participant Briefing

What this challenge is, what you receive, and what you are expected to do.

Code Craft Society • Reverse Engineering Hackathon • 15–16 October 2026

1. What is this challenge about?

Most hackathons give you a problem statement and ask you to build something new. **Re-InnovateX works the other way round.** You are given a system that already existed, but it was damaged: its source code is gone, nobody wrote down how it worked, and the version you receive is only a **recovered build**. It still runs, but parts of it are wrong.

Your team's job is to work out **what the system is, what it is for, and how it really behaves**, and then to rebuild it properly. Nobody will tell you the problem statement. Discovering it is the first part of the challenge.

Understand → Deconstruct → Reconstruct → Innovate → Demo

This is a team challenge (2–4 members). All teams work from the same rules, but **teams may be given different systems**, so do not assume your neighbours are solving the same problem.

2. What you receive

Item	What it is
A recovered build (one HTML file)	A web app that opens in any browser and works offline. Its screens, data and controls are real, but one of its modules (the "engine" that makes decisions) is unreliable.
The system's own data	Records, reference tables and requests inside the app, visible in its Registry tab.
Recovered history	The History tab holds results and logs from the *original* system. This is your **ground truth**: it shows what the real system did.
A short brief (README)	Your mission and the exact output format you must produce for the hidden test.

You do **not** receive the original source code, a specification, or the answers.

Important: the history records are trustworthy. The recovered build's own results are **not**. Where they disagree, the history wins.

3. How to explore the recovered build

The app has the same five areas in every challenge:

- **Registry** — the system's reference data and the records it works on. Read it carefully: what kinds of things exist, and how are they described?
- **New Entry** — a form for adding a record. The fields tell you what the system needs to know.
- **The engine tab** — runs the system's decision logic on the records and shows its output. You can load other data files and export the result. *This is the part that is broken.*
- **A fourth tab** — extra information the system keeps (a queue or a calendar). It may look empty or odd until the engine works properly.
- **History** — recovered results and logs from the original system.

You may use your browser's developer tools to inspect the page. Everything the file exposes is fair to study.

4. What you are supposed to do

Phase 1 — Understand and observe

- Work out **what the system is for**, who uses it, and what a "good" result looks like.
- Run it. Compare what it does with what the history says it did. **Where does it differ, and how often?**
- Keep an observation log from the start: inputs, outputs, errors, surprises.
- Write a one-paragraph problem statement in your own words.

Phase 2 — Deconstruct

- Infer the **real rules** behind the system's decisions: thresholds, limits, special cases, and **the order in which rules are applied**.
- Check what **earlier records do to later ones** (some outcomes depend on what happened before).
- Look for records that sit just either side of a limit. That is where rules show themselves.
- For every rule you claim, note the **evidence** (which records support it). Separate **facts** from **guesses**.

Phase 3 — Reconstruct

- Rebuild or repair the system so that it reproduces the original behaviour. You may fix the provided HTML or build your own version in any language, as long as you meet the output format.
- Test it against the history records and your own edge cases. Keep backups of working versions.
- **Get the baseline working before you try to innovate.**

Innovation

Once the baseline works, add something meaningful: for example an explanation of *why* each decision was made, input validation, a test suite, a dashboard, better usability or accessibility, or automation. Keep it clearly **separate** from the baseline, and make sure it does not break it.

5. What you submit

1. **The working system** and a note on how to run it.
2. **A short write-up** with these sections: System Understanding, Failure Analysis, Deconstruction (your inferred rules with evidence), Reconstruction, Testing, Innovation, Limitations, Demo Guide.
3. **A 5-minute demo** on Day 2, where every team member can answer questions.

The hidden test

At the end of the build time, organizers will give you a **hidden data file** in the same format as the system's own data. Your system must produce its decisions for that file in the output format given in your README. The result is scored automatically and is part of your Reconstruction marks. Matching the sample records is **not** enough: the hidden file contains cases the samples do not cover, so rules must be inferred correctly.

6. Suggested time plan (about 9 hours)

Block	Time	Goal
Briefing	0:30	Rules, restrictions, kit hand-out
Understand and observe	1:30	State what the system is and what is broken
Deconstruct	2:00	Rules inferred, each with evidence
Reconstruct	2:30	Baseline working and tested
Innovate	1:00	One meaningful improvement
Documentation, hidden test, submission	1:30	Write-up, demo guide, final output

The organizers will confirm the exact timings and checkpoints on the day.

7. How you will be judged

Criterion	Weight	What judges look for
Understanding	20%	Accurate observations; you can say what the system is for
Technical Analysis	20%	Correct, evidence-backed rules, including their order and limits
Reconstruction	30%	Correct, stable, tested rebuild; hidden-test result
Innovation	20%	Meaningful, demonstrable improvement beyond the baseline
Documentation and Demo	10%	Clear reasoning, honest limitations, reproducible demo

Judges care about **reasoning, not only output**. They will ask different team members to explain decisions, so everyone should understand the whole solution.

8. Rules to remember

- Work only with the files provided. Do not look for the original source code, leaked solutions or other teams' work.
- Stay inside the challenge environment. If you find an unintended way in or out, **stop and tell an organizer**.
- Use of AI tools and the internet follows the **instructions announced at the briefing**. If a restriction is announced, it is binding. If you are unsure, ask first.
- Do not claim a feature you cannot demonstrate. Say honestly what does not work.
- Be respectful to other teams, organizers and judges. Organizer and judge decisions are final.

9. Habits that win

- **Observe before assuming.** Evidence first, then a hypothesis.
- **Test boundaries.** Exactly at a limit, one above, one below.
- **Check the order.** When two rules both apply, one of them wins. Find out which.
- **Write as you go.** Do not rebuild your story at the end.
- **Back up** every working version.

10. Quick checklist

☐ Opened the recovered build and explored all five areas

☐ Wrote down what the system is for

☐ Compared the build's output with the history and listed the differences

☐ Listed the inferred rules, each with evidence

☐ Baseline reproduces the history records

☐ Tested edge cases and kept backups

☐ Innovation added without breaking the baseline

☐ Write-up and demo guide finished

☐ Hidden-test output produced in the required format

☐ Every team member can explain the solution

11. Contacts

Role	Name	Contact
Faculty Coordinators	Dr. Geetanjali Surange, Mr. Shubham Dhakarey	—
Faculty Co-Coordinator	Mr. Vipin Jadon	—
Student Coordinator	Garvit Mishra	+91 75809 89439
Student Co-Coordinator	Ujjwal Gupta	+91 91110 16526

Re-InnovateX: Don’t just use technology. Understand it. Break it. Rebuild it.